

deepreview-skill

Tutoriel utilisateur

Version 0.2.1

Mai 2026

Skill multi-agents pour Claude Code : revue pré-merge (**deepreview**) et audit de code existant (**deepaudit**), avec étape de reproduction indépendante des findings.

github.com/aiskool/deepreview-skill

Licence MIT

1. Introduction

deepreview-skill regroupe deux skills complémentaires pour Claude Code, conçues pour produire des analyses de code rigoureuses grâce à une architecture multi-agents.

deepreview — revue pré-merge d'une diff git (branche locale, PR GitHub, ou ref de comparaison explicite). À utiliser avant de merger une feature branch.

deepaudit — audit de code existant (un dossier précis ou un axe transversal de tout le repo). À utiliser pour identifier ce qui ne va pas dans une codebase déjà déployée.

Les deux suivent le même schéma : six agents spécialisés en parallèle (sécurité, architecture, bugs, performance, couverture de tests, documentation), suivis d'un agent *verifier* qui reproduit chaque finding indépendamment avant de le rapporter. Les findings non reproductibles sont supprimés du rapport final — c'est ce qui donne au rapport son ratio signal/bruit.

Encadrement honnête. Cette skill s'inspire du pattern architectural documenté de la commande Claude Code `/ultrareview` (multi-agents + étape de reproduction). Ce n'est pas un portage de l'implémentation interne d'Anthropic, qui n'est pas publique. Tout tourne localement dans votre session Claude Code, pas dans un sandbox distant.

Quand utiliser quelle skill

Situation	Skill
Branche feature avant merge sur main	deepreview
Audit d'une PR GitHub	deepreview
Reprise d'un projet existant	deepaudit
Audit ciblé d'un sous-système (auth, paiement)	deepaudit <chemin>
Audit d'un seul axe (sécurité partout)	deepaudit <axe>
Commit court à valider rapidement	/review natif suffit

2. Prérequis

Avant d'installer la skill, vérifiez les prérequis suivants.

- **Claude Code installé** et fonctionnel. Version 2.1.86 ou supérieure recommandée.
- **Un dépôt git**. Les deux skills opèrent sur du code tracké par git et créent un checkpoint commit avant la phase de vérification.
- **Le test runner du projet** doit être installé pour les axes exécutables (bugs, performance, tests). La skill détecte automatiquement npm/yarn/pnpm/bun, pip/poetry/uv, cargo, go, bundler, maven/gradle, composer, mix, dotnet.
- **macOS, Linux, ou Windows**. Un installer Bash (`install.sh`) couvre macOS, Linux et Windows avec WSL ou Git Bash. Un installer PowerShell (`install.ps1`) couvre Windows natif. Voir la section 3 pour le détail.

Bash reste requis à l'exécution, même sur Windows. Le détecteur de runtime et plusieurs exemples du SKILL.md font appel à bash. Sur Windows natif, assurez-vous que bash est dans le PATH via WSL (recommandé), Git for Windows, Cygwin, ou MSYS2. Sans bash, les axes exécutables (bugs, performance, tests) basculent en mode statique.

Si votre stack n'est pas reconnue par le détecteur de runtime, les axes statiques (sécurité, architecture, documentation) fonctionnent quand même ; les axes exécutables basculent en mode vérification statique uniquement.

3. Installation

Deux installers sont fournis. Les deux respectent le même protocole de variables d'environnement (`DEEPPREVIEW_SCOPE`, `DEEPPREVIEW_REF`, `DEEPPREVIEW_REPO`, `DEEPPREVIEW_SKILLS`) et produisent une arborescence identique sous `.claude/`.

macOS, Linux, ou Windows avec WSL/Git Bash

Le format en deux temps permet de relire le script avant exécution. C'est le geste de sécurité standard pour tout `curl | bash` et c'est ce que vous devez communiquer à vos utilisateurs.

```
curl -fsSL https://raw.githubusercontent.com/aiskool/\
  deepreview-skill/main/install.sh -o /tmp/deepreview-install.sh
less /tmp/deepreview-install.sh  # à lire avant d'exécuter
DEEPPREVIEW_REF=v0.2.1 bash /tmp/deepreview-install.sh
```

Cette commande installe les **deux skills** (deepreview + deepaudit) dans le dossier `.claude/` du dépôt git courant.

Variantes (Bash)

Globale — disponible dans tous vos projets :

```
DEEPPREVIEW_SCOPE=global DEEPPREVIEW_REF=v0.2.1 \
  bash /tmp/deepreview-install.sh
```

Une seule des deux skills :

```
DEEPPREVIEW_SKILLS=deepreview DEEPPREVIEW_REF=v0.2.1 \
  bash /tmp/deepreview-install.sh
```

```
DEEPPREVIEW_SKILLS=deepaudit DEEPPREVIEW_REF=v0.2.1 \  
bash /tmp/deepreview-install.sh
```

Windows (PowerShell)

Même schéma review-then-run, en utilisant Invoke-RestMethod au lieu de curl :

```
irm https://raw.githubusercontent.com/aiskool/  
deepreview-skill/main/install.ps1 -OutFile install.ps1  
notepad install.ps1 # a lire avant d'executer  
$env:DEEPPREVIEW_REF = "v0.2.1"  
.\install.ps1
```

Si la politique d'exécution PowerShell bloque le script, lancez-le une fois en bypass :

```
powershell -ExecutionPolicy Bypass -File install.ps1
```

Variantes (PowerShell)

```
# Installation globale  
$env:DEEPPREVIEW_SCOPE = "global"  
$env:DEEPPREVIEW_REF = "v0.2.1"  
.\install.ps1  
  
# Une seule skill  
$env:DEEPPREVIEW_SKILLS = "deepreview"  
$env:DEEPPREVIEW_REF = "v0.2.1"  
.\install.ps1
```

Pinning de version

Le DEEPPREVIEW_REF=v0.2.1 (ou \$env:DEEPPREVIEW_REF="v0.2.1") cible une release immutable. À chaque nouvelle release publiée sur le repo, mettez à jour cette valeur. Sans pinning explicite, le script utilise main, qui peut changer.

***Note importante** : install.ps1 est servi depuis main uniquement. Le pinning v0.2.1 s'applique au contenu des skills installées, pas au script d'installation lui-même.*

Vérifier l'installation

Sur shell Bash :

```
ls .claude/skills/ # deepaudit/ deepreview/  
ls .claude/agents/ | grep -E "auditor|reviewer|verifier"  
# doit afficher 14 lignes (7 auditors + 7 reviewers/verifier)
```

Sur PowerShell :

```
Get-ChildItem .claude\skills\  
Get-ChildItem .claude\agents\ | Where-Object Name -match `  
'auditor|reviewer|verifier'
```

Redémarrez Claude Code pour que les nouveaux sub-agents soient chargés. Dans une session active, tapez /agents et vérifiez la liste.

4. Utiliser deepreview

Trois modes d'invocation

```
deepreview                                # diff vs branche par défaut
deepreview PR 1234                        # PR GitHub spécifique
deepreview against origin/develop        # diff vs un autre ref
```

Ce qui se passe pendant un run

Phase 1 — Scope. Claude détermine la diff à analyser, détecte le runtime du projet, construit le manifeste des fichiers concernés.

Phase 2 — Fan-out. Six agents reviewers partent en parallèle (security, architecture, bug-hunter, performance, test-coverage, documentation). Chacun lit la diff et produit un JSON de findings.

Phase 3 — Vérification. Un checkpoint git est créé. Pour chaque finding, un agent verifier tente de le reproduire — soit par un test écrit (axes exécutables), soit par walkthrough statique (sécurité, architecture, docs). Les findings non reproductibles sont marqués unverified.

Phase 4 — Synthèse. Findings dédoublés, triés par sévérité (critical / important / nice-to-have), rapport markdown généré.

Phase 5 — Lessons. Les patterns critical sont appendés à `.claude/lessons.md`.

Phase 6 — Cleanup. Le rapport est sauvegardé dans `.claude/review-reports/<run-id>-<scope-hint>.md`. Le dossier de travail temporaire est supprimé.

Durée typique : 5 à 15 minutes selon la taille de la diff.

5. Utiliser deepaudit

Cinq formes d'invocation

deepaudit	# REFUSÉ : scope requis
deepaudit src/auth/	# 6 auditeurs sur src/auth/
deepaudit security	# 1 auditeur sur tout le repo
deepaudit security src/auth/	# 1 auditeur sur src/auth/
deepaudit src/auth/ src/billing/	# 6 auditeurs sur l'union

Axes valides : security, architecture, bugs, performance, tests, docs.

Synonymes acceptés : bug-hunter → bugs, test-coverage → tests, documentation → docs.

Pourquoi un scope est requis

L'audit non-scopé sur un monorepo produit un rapport ingérable et coûte beaucoup en contexte. La skill refuse explicitement deepaudit sans argument et vous demande de préciser. C'est volontaire.

Configuration

DEEPAUDIT_MAX_FINDINGS (défaut : 20) limite la taille du rapport final. Toujours rempli en priorité par les findings critical, complété par les important si reste de la place. Les nice-to-have sont systématiquement supprimés.

```
DEEPAUDIT_MAX_FINDINGS=30 deepaudit security backend/
```

Top-N synthesis

Si 50 findings critical sont vérifiés, seuls les 20 (ou la valeur configurée) premiers sont rapportés. Le rapport indique explicitement combien d'autres existent au-delà du cap, par axe et par sévérité. C'est ce qui empêche un audit large de produire un rapport illisible.

6. Lire un rapport

Localisation

Après chaque run, le rapport est préservé à un emplacement stable.

Skill	Chemin
deepreview	.claude/review-reports/<run-id>-<scope-hint>.md
deepaudit	.claude/audit-reports/<run-id>-<scope-hint>.md

Le <run-id> est un horodatage UTC (20260506-114333). Le <scope-hint> est dérivé automatiquement de l'invocation.

Structure d'un rapport

```
# deepaudit — 20260506-114333

Scope: backend/src/middleware/
Axes: security
Files: 8

3 critical · 4 important · 0 nice-to-have

## Critical
### security · backend/src/middleware/file.ts:44-107
[titre du finding]
[description]
Reproduction: [evidence]
Suggested fix: [actionable change]

## Important
[...]
```

Sévérités

critical — peut casser la production, exfiltrer des données, bypasser l'authentification, ou corrompre l'état persistant.

important — comportement incorrect réaliste, vulnérabilité exploitable sous conditions, dette technique mesurable.

nice-to-have — amélioration suggérée. **Supprimée** du rapport final pour deepaudit (le top-N ne contient que des critical et des important).

Findings vérifiés

Chaque finding qui apparaît dans le rapport a été indépendamment reproduit :

- Pour les axes **exécutables** (bugs, performance, tests) : un test de reproduction a été écrit et exécuté.
- Pour les axes **statiques** (security, architecture, docs) : un walkthrough rigoureux confirme la chaîne d'exploitation ou la violation.

Les findings dont la reproduction échoue sont **filtrés** du rapport. Cela réduit drastiquement le bruit par rapport à un linter classique.

7. Sécurité et garanties

Ce que la skill NE touche PAS

- Aucun fichier source du projet n'est modifié par les agents reviewers ou auditors.
- Le verifier peut écrire des tests de reproduction, mais **uniquement** dans le dossier de travail `<run-id>/`.
- Un checkpoint git est créé avant la phase de vérification. À la fin du run, le working tree est restauré à ce checkpoint.
- `.claude/lessons.md` est partagé avec d'autres skills (Autopilot, etc.). La cleanup phase ne le touche **jamais**.
- Le verifier n'exécute **jamais** de payload d'exploit. La vérification sécurité est intellectuelle, pas active.

Ce qui est nettoyé après chaque run

Le dossier de travail `.claude/audit/<run-id>/` (ou `.claude/review/<run-id>/`) est supprimé après que le rapport a été préservé. Plus d'accumulation au fil des runs.

Ce que vous gardez

- Les rapports dans `.claude/audit-reports/` et `.claude/review-reports/` (durables)
- `.claude/lessons.md` (partagé, jamais supprimé)
- L'historique git du projet (intouché, sauf les checkpoint commits qui sont reset à la fin)

8. Limites honnêtes

Ce que cette skill n'est pas

Pas un remplacement de /ultrareview. Si vous avez accès et le budget, la version cloud d'Anthropic a plus de compute et un sandbox distant. Cette skill tourne localement avec les contraintes de votre fenêtre de contexte.

Pas un remplacement de la CI. Linters, type checkers, suite de tests doivent toujours tourner où ils tournent déjà. La skill ajoute une couche sémantique en complément.

Pas un remplacement de la review humaine. Un reviewer qui connaît le produit, les utilisateurs, et la roadmap attrape ce qu'aucun agent ne peut deviner.

Coût en contexte

Sur un audit complet, six agents en parallèle peuvent consommer 300k+ tokens de contexte. Sur des dépôts très grands ou avec un budget limité, restreignez le scope (un dossier précis, ou un seul axe).

Faux positifs

Aucun verifier n'est parfait. Sur les runs typiques, comptez 5 à 15 % de findings discutables (mauvaise interprétation du flow, pattern correctement détecté mais sévérité surnotée, etc.). Lisez le rapport critique en main, ne traitez pas chaque finding comme un bug avéré.

Variabilité

Deux runs sur la même surface peuvent produire des rapports différents (les agents sont non-déterministes). Pour des décisions importantes, croisez avec une revue humaine.

9. Bonnes pratiques

Premier audit d'un projet inconnu

Commencez petit. Ne lancez pas `deepaudit` sur tout le repo en première intention — vous serez submergé. Choisissez un sous-système où vous avez un soupçon précis :

```
deepaudit security backend/src/auth/  
deepaudit backend/src/payment/
```

Avant un release ou un push significatif

Lancez `deepreview` sur la branche `feature`, lisez le rapport, traitez les `criticals` avant le `merge`. Le coût (5-15 minutes) est négligeable comparé au coût d'un bug en production.

Suivi des findings au fil du temps

Les rapports dans `.claude/audit-reports/` sont datés. Au fil du temps, vous pouvez :

- Comparer un nouveau rapport au précédent pour voir ce qui a été corrigé.
- Garder un journal de progression (transformer les findings en tickets ou issues GitHub).
- Identifier les patterns récurrents dans `lessons.md` et les corriger structurellement.

10. Dépannage

« `deepaudit needs a scope` »

Comportement attendu — vous avez tapé `deepaudit` sans argument. Précisez un dossier, un axe, ou les deux. Voir section 5.

`RUN_ID` introuvable / chemins cassés

Le run a été interrompu en cours de route. Les marqueurs `/tmp/deep*_*.txt` peuvent rester. Pour repartir d'une base propre :

```
rm -f /tmp/deepaudit_* /tmp/deepreview_*
```

Le test runner échoue avant la vérification

Si la baseline (suite de tests sur le code actuel) est cassée, le `verifier` ne peut pas distinguer une régression introduite par un bug du finding qu'il essaie de reproduire. La `skill` le détecte et bascule en mode `static-only` pour les axes concernés. Solution durable : réparez votre baseline.

Le rapport est vide

Soit aucun finding n'est passé la vérification (votre code est très propre, peu probable), soit le scope était trop étroit, soit la baseline était cassée. Élargissez le scope ou inspectez les logs du run précédent.

11. Liens et ressources

Ressource	URL
Repo public	github.com/aiskool/deepreview-skill
Releases	github.com/aiskool/deepreview-skill/releases
Issues / bug reports	github.com/aiskool/deepreview-skill/issues
Security advisories	github.com/aiskool/deepreview-skill/security/advisories
Guide de contribution	Voir CONTRIBUTING.md du repo
Licence	MIT (voir LICENSE)

Reporter une vulnérabilité

Si vous découvrez une faille de sécurité dans la skill (par exemple une injection dans la logique d'orchestration, une écriture non intentionnelle hors du dossier de run, ou un moyen de faire exécuter du code arbitraire au verifier), **n'ouvrez pas une issue publique**. Ouvrez plutôt un security advisory privé via GitHub à l'URL listée ci-dessus.

Mises à jour

Pour mettre à jour vers une nouvelle release, relancez l'installation avec le nouveau DEEPREVIEW_REF et redémarrez Claude Code. Les agents sont chargés au démarrage de session.

Bonne utilisation. La skill est meilleure quand vous l'utilisez avec discernement — un audit n'est pas une todo-list, c'est un point de départ pour la conversation avec le code.